

HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY AND EDUCATION

Advanced Master Program - DDM501

SYSTEM SPECIFICATION DOCUMENT (SPS)

Real-Time Fraud Detection ML System

End-to-End Solution for Risk Scoring & Decision Intelligence

Team Members:

1. Duong Binh An (25MSA23234)
2. Nguyen Le Hong Nhi (25MSA23235)
3. Le Quang Tuyen (25MSA23232)

Instructor:

PhD Huynh Cong Viet Ngu

October 2026
TPHCM, Vietnam

“Fraud detection is not classification; it is decision-making under constraints.”

Contents

1 Executive Summary

1.1 Purpose

This document specifies the complete end-to-end fraud detection system that:

- Scores transactions using machine learning (uncalibrated risk ranking)
- Maps scores to decisions using explicit tiered thresholds (LOW/REVIEW/HIGH)
- Serves predictions in real-time via FastAPI with observability
- Visualizes decisions through an interactive frontend dashboard
- Monitors performance with Prometheus metrics and Grafana dashboards
- Deploys reproducibly using Docker and Docker Compose

1.2 Key Design Principle

Fraud detection is not classification; it is decision-making under constraints.

The system explicitly separates:

1. **Risk Scoring** (ML model) → uncalibrated ranking signal $[0,1]$
2. **Decision Intelligence** (Policy layer) → tiered actions bound by operational capacity

1.3 Core Deliverables

- Training pipeline with data validation and model selection
- FastAPI prediction service with schema validation
- Prometheus metrics and Grafana dashboards
- Frontend dashboard for real-time decision visualization
- Docker Compose deployment stack
- Unit and integration tests (80% coverage gate)
- Responsible AI documentation (fairness, explainability, privacy)

2 Problem Definition and Business Context

2.1 Business Problem

Payment card fraud produces:

- **Direct costs:** Chargebacks, reimbursed losses
- **Indirect costs:** Manual review labor, customer support, churn due to friction

A fraud scoring service reduces losses by:

1. **Identifying** risky transactions early (before settlement or chargeback)
2. **Prioritizing** reviewer capacity (manual checks on highest-value/highest-risk)
3. **Enabling** tiered actions (allow low-risk, review medium-risk, block high-risk)

2.2 Personas and Use Cases

Persona	Goal	Interaction
Risk Operations Analyst	Prioritize manual review	View score, tier, explainability
Backend/ML Engineer	Deploy and monitor	Deploy via Compose; monitor metrics
Compliance/Audit	Verify decision consistency	Model metadata, thresholds, SHAP

3 System Architecture

3.1 High-Level Overview

The system covers the complete ML lifecycle:

DATA → PIPELINE → MODEL → API → FRONTEND → MONITORING →
DEPLOYMENT

3.1.1 Key Components

1. **Data Layer:** CSV ingestion, schema validation, sampling
2. **ML/Training Layer:** Pipeline orchestration, model training, threshold tuning
3. **Serving Layer:** FastAPI inference service, streaming simulation
4. **Presentation Layer:** Real-time dashboard UI
5. **Observability Layer:** Prometheus metrics export
6. **DevOps/Deployment Layer:** Docker, Docker Compose orchestration
7. **QA/Testing Layer:** Unit, integration, data quality tests
8. **CI/CD Layer:** GitHub Actions automation

3.2 Component Responsibilities Mapping

Folder	Layer	Responsibility	Tech
src/data/	Data	Loading, validation, sampling	Pandas, NumPy
src/pipelines/	ML/Training	Pipeline, training, thresholds	scikit-learn, LightGBM
src/api/	Serving	HTTP API, schema validation	FastAPI, Pydantic
frontend/	Presentation	Real-time dashboard	HTML/CSS/JavaScript
src/monitoring/	Observability	Prometheus metrics	prometheus-client
deployment/	DevOps	Docker, Compose, Monitoring	Docker, Prometheus, Grafana
tests/	QA	Unit, integration tests	pytest
.github/workflows/	CI/CD	Test gates, Docker builds	GitHub Actions

4 Functional Requirements

4.1 Core Prediction Requirements

ID	Requirement	Prio	Verification
FR-1	Accept transaction features, return risk score, tier, and action	MUST	POST /predict endpoint
FR-2	Document risk score as uncalibrated (not calibrated probability)	MUST	score_semantics field in API
FR-3	Map risk scores to decision tiers using explicit thresholds	MUST	model_info.json metadata
FR-4	Determine action: LOW→allow, REVIEW→review, HIGH→block	MUST	src/api/main.py line 158-163

4.2 Data and Pipeline Requirements

ID	Requirement	Prio	Verification
----	-------------	------	--------------

FR-5	Ingest Kaggle Credit Card Fraud dataset, validate schema	MUST	data/archive/creditcard.csv
FR-6	Split into train/val/test (70/15/15) with stratification	MUST	split_info.json
FR-7	Train baseline (Logistic Regression) and improved (LightGBM)	MUST	Comparison in model_selection_summary.json
FR-8	Perform threshold tuning for top-K review-rate policy	MUST	Thresholds in model metadata
FR-9	Generate reproducible model artifacts and metadata	MUST	artifacts/models/ and reports/

4.3 API Endpoint Requirements

ID	Requirement	Prio	Verification
FR-10	POST /predict: Accept features or features_by_name	MUST	src/api/main.py
FR-11	GET /health: Return model status and metadata	MUST	src/api/main.py
FR-12	GET /metrics: Export Prometheus metrics	MUST	GET /metrics endpoint
FR-13	GET /features/schema: Return feature count and names	MUST	Endpoint implemented
FR-14	GET /stream/pull: Return paginated scored events	MUST	Demo streaming

FR-15	Return HTTP 422 for malformed requests	MUST	Pydantic validation
FR-16	Return HTTP 503 if model not loaded	MUST	Model loading check
FR-17	Include unique request_id in all responses	SHOULD	src/utls/ids.py

4.4 Monitoring and Observability

ID	Requirement	Prio	Verification
FR-18	Emit Prometheus metrics (request count, latency, predictions)	MUST	src/monitoring/metrics.py
FR-19	Prometheus scrape from /metrics every 5 seconds	MUST	prometheus.yml
FR-20	Grafana display dashboards showing metrics	MUST	fraud_api.json dashboard
FR-21	Prometheus evaluate alert rules	SHOULD	alerts.yml

5 Non-Functional Requirements

5.1 Performance

- Single prediction latency (p95): 500 ms
- Model loading time on startup: 5 seconds
- Throughput: 10 requests/sec on laptop-class hardware
- Prometheus scrape interval: 5 seconds

5.2 Reliability

- API uptime under steady load: 99%
- Error rate (5xx): < 5%
- Feature mismatch validation: HTTP 422 with clear message
- Graceful degradation on model load failure: HTTP 503

5.3 Scalability

- Horizontal scaling via containerization
- Stateless API design (no session state)
- Artifact storage decoupled from API

5.4 Maintainability

- Modular architecture with clear layer separation
- Type hints and docstrings throughout
- Reproducible model training with versioning
- Clear API contracts via Pydantic schemas

5.5 Security & Privacy (Demo Scope)

Requirement	Current State	Future Work
No PII in inputs	Vector of 30 features	N/A
No raw features in logs	Policy documented	Enforce filtering
API authentication	Not implemented	Add JWT/API key
Rate limiting	Not implemented	Add slowapi
Model artifact signing	Not implemented	Sign joblib artifacts
Audit logging	Not implemented	Log predictions with metadata

6 Data Design and Dataset Analysis

6.1 Dataset Overview

Property	Value
Source	Kaggle Credit Card Fraud Detection
Location	data/archive/creditcard.csv
Rows	284,807
Columns	31 (30 features + target)

6.2 Schema

- Time (int): Seconds within transaction period
- V1-V28 (float): PCA-transformed anonymized features

- **Amount** (float): Transaction amount (currency)
- **Class** (int): 0 = legitimate, 1 = fraud

6.3 Class Distribution (Severe Imbalance)

- Total: 284,807 rows
- Legitimate (0): 284,315 (99.83%)
- Fraud (1): 492 (0.17%)
- Fraud rate: 0.17% — extremely imbalanced

6.4 Data Validation Pipeline

1. Schema check: Verify columns present and types correct
2. Missing values: Count and report per column
3. Duplicates: Flag exact duplicate rows
4. Statistical summary: Min, max, mean for each feature
5. Class distribution: Count and log imbalance ratio
6. Output: artifacts/reports/dataset_schema.json, missing_values.csv

6.5 Data Splitting

- Ratio: 70% train / 15% validation / 15% test
- Method: Stratified random (preserves fraud rate)
- Stratification column: Class (fraud/non-fraud)
- Random seed: 42 (configurable, for reproducibility)
- Output: artifacts/reports/split_info.json

7 ML Pipeline Design

7.1 Pipeline Orchestration

The complete training workflow is implemented in `src/pipelines/run_model_workflow.py`:

1. Load & Validate Dataset
2. Train/Val/Test Split (70/15/15, stratified)
3. Preprocess (scaling via StandardScaler)
4. Train Baseline (Logistic Regression)

5. Train Improved Model (LightGBM with hyperparameter grid)
6. Evaluate Both Models on Validation Split
7. Threshold Tuning (Top-K policy)
8. Model Selection (Primary: Val PR-AUC; Tie-break: Recall)
9. Generate Figures (ROC, PR curves, confusion matrices)
10. Export Artifacts (Model, metadata, reports)
11. Log to MLflow

7.2 Baseline Model: Logistic Regression

Configuration:

```
LogisticRegression(  
    class_weight='balanced',  
    max_iter=1000,  
    random_state=seed,  
    solver='lbfgs'  
)
```

Within Pipeline:

- Wrapped in scikit-learn Pipeline([StandardScaler(), LogisticRegression(...)])
- Scaling applied to train; fitted scaler applied to val/test
- Output: baseline_logistic_regression_pipeline.joblib

7.3 Improved Model: LightGBM

Hyperparameter Grid:

```
param_grid = {  
    'learning_rate': [0.01, 0.03, 0.05],  
    'n_estimators': [200, 300, 450],  
    'num_leaves': [31, 63, 127],  
    'scale_pos_weight': [10.0, 20.0, 50.0]  
}
```

7.4 Evaluation Metrics

Metric	Purpose	Interpretation
PR-AUC	“Primary“ rank- ing quality	[0,1]; higher better; robust to imbalance
ROC-AUC	Secondary rank- ing metric	[0,1]; less sensitive to imbalance
Precision	False positive cost	$TP/(TP+FP)$; high at high thresholds
Recall	False negative cost	$TP/(TP+FN)$; high recall at low thresh- olds
F1 Score	Harmonic mean	Trade-off metric

8 Model Training, Evaluation, and Selection

8.1 Training Results

8.1.1 Baseline Model (Logistic Regression)

Operating Point	Threshold	Precision	Recall	PR-AUC
Review (top 1%)	0.7391	0.147	0.851	0.630
High (top 0.2%)	0.9999	0.663	0.770	0.630

8.1.2 Improved Model (LightGBM)

Operating Point	Threshold	Precision	Recall	PR-AUC
Review (top 1%)	0.0000004	0.133	0.770	0.629
High (top 0.2%)	0.0003	0.640	0.743	0.629

8.2 Selection Decision

Selected Model: Logistic Regression

Rationale:

1. Primary criterion (Val PR-AUC): Baseline 0.630 > Improved 0.629 (marginal win)
2. Tie-breaker (Recall at REVIEW): Baseline 0.851 > Improved 0.770 (significant)
3. Interpretability: Baseline provides coefficients for explainability
4. Stability: Simpler model, lower risk of overfitting

8.3 Test Set Evaluation

Metric	Value
PR-AUC	0.769
ROC-AUC	0.965
Review Tier Precision	0.146
Review Tier Recall	0.851
High Tier Precision	0.843
High Tier Recall	0.797

Inference: Model generalizes well to unseen test data; no evidence of severe overfitting.

9 Threshold Strategy and Decision Logic

9.1 Top-K Capacity-Driven Policy

Instead of metric optimization, thresholds are set to match operational review capacity:

Tier	Threshold	Top-K Rate	Condition
LOW	—	—	<code>score < threshold_review</code>
REVIEW	0.7391	1%	<code>threshold_review ≤ score < threshold_high</code>
HIGH	0.9999	0.2%	<code>score ≥ threshold_high</code>

9.2 Operational Benefit

- **Predictable queue size:** If 10K trans/day, expect 100 review + 20 block cases
- **No data-dependency tuning:** Percentile holds if fraud rate changes
- **Defensible policy:** “Flag the top 1% riskiest transactions for review”

9.3 Example

If system processes 1,000,000 transactions/day:

- 10,000 flagged for REVIEW (1%)
- 2,000 flagged for HIGH/BLOCK (0.2%)
- 988,000 allowed automatically

9.4 Decision Logic in API

Location: `src/api/main.py` lines 158-163

```
def _tier_and_action(score, *, threshold_review, threshold_high):
    if score >= threshold_high:
        return "HIGH", "block"
    if score >= threshold_review:
        return "REVIEW", "review"
    return "LOW", "allow"
```

9.5 Risk Score Semantics (Critical)

Claim: Risk score is NOT a calibrated probability of fraud

Why:

- No calibration validation (no Brier score, calibration curves)
- Imbalance weighting changes learning objective
- Random split (not time-aware) → potential dataset shift
- Thresholds are percentile-based (ordinal ranking), not probabilistic

What it IS:

- Uncalibrated ranking signal
- Higher score → higher model confidence
- Use for prioritization and tiering, not “60% chance of fraud” claims

Communicated to users:

- API response includes `score_semantics`: `"risk_score_uncalibrated"`
- Frontend disclaimer displayed
- Documentation clearly states this limitation

10 API Specification

10.1 Endpoints Overview

Endpoint	Method	Purpose
/predict	POST	Score a transaction
/health	GET	System status and metadata
/metrics	GET	Prometheus metrics export
/features/schema	GET	Feature count and names
/stream/pull	GET	Simulated event stream (demo)

10.2 POST /predict

Request:

```
{
  "features": [10.0, -1.3, 1.5, ...], // 30 floats
  "features_by_name": null
}
```

Response (200 OK):

```
{
  "request_id": "req-12a34b56...",
  "risk_score": 0.85,
  "risk_tier": "REVIEW",
  "action": "review",
  "threshold_review": 0.7391,
  "threshold_high": 0.9999,
  "score_semantics": "risk_score_uncalibrated",
  "model_version": "20260415T094316Z"
}
```

Error Responses:

- 422 Unprocessable Entity: Feature count mismatch
- 503 Service Unavailable: Model not loaded

10.3 GET /health

Returns model status and metadata:

```
{
  "status": "ok",
  "model_loaded": true,
  "model_version": "20260415T094316Z",
  "expected_features": 30,
  "threshold_review": 0.7391,
  "threshold_high": 0.9999,
  "score_semantics": "risk_score_uncalibrated"
}
```

10.4 GET /metrics

Prometheus text format with:

- `api_requests_total`: Request count by endpoint and status
- `api_request_latency_seconds`: Latency histogram
- `fraud_predictions_total`: Predictions by tier
- `fraud_actions_total`: Actions by type

11 Frontend System Design

11.1 Architecture

Framework: Vanilla JavaScript (no React/Vue) — lightweight, zero build step

Main files:

- `index.html`: Static structure

- `styles.css`: Responsive styling
- `app.js`: Page initialization, state management
- `api-client.js`: API wrapper (fetch calls)
- `ui.js`: DOM updates and rendering

11.2 Key Features

11.2.1 Dashboard View

Section	Content
Header	Model version, thresholds, score semantics
Control Panel	API URL, stream mode, speed, start/stop
Live Summary KPIs	Total trans., fraud count, avg score
Transactions Table	Live feed with risk tier coloring
Alerts Panel	HIGH/REVIEW tier events

11.2.2 Real-Time Updates

```
// Polling every N milliseconds
setInterval(() => {
  const response = await fetch(`${apiBaseUrl}/stream/pull`);
  const data = response.json();
  updateDashboard(data.events);
}, speed_ms);
```

11.3 Visualizations

- **Status Pill**: Green (connected), Yellow (connecting), Red (disconnected)
- **Risk Tier Coloring**: GREEN=LOW, YELLOW=REVIEW, RED=HIGH
- **KPI Cards**: Fraud count, review count, average score

12 Monitoring and Alerting

12.1 Prometheus Metrics

Scrape Configuration:

- Target: `http://api:8000/metrics`
- Scrape interval: 5 seconds
- Configured in: `deployment/prometheus/prometheus.yml`

12.1.1 Key Metrics

Metric	Type	Purpose
api_requests_total	Counter	Request volume by endpoint/status
api_request_latency_seconds	Histogram	Latency distribution
fraud_predictions_total	Counter	Predictions by tier
fraud_actions_total	Counter	Actions (allow/review/block)

12.2 Grafana Dashboards

Location: `deployment/grafana/dashboards/fraud_api.json`

Key panels:

- Request rate (line chart)
- Error rate % (gauge)
- p95 latency (gauge)
- Prediction distribution (pie chart)
- Model uptime (counter)

12.3 Alert Rules

Location: `deployment/prometheus/alerts.yml`

Example rules:

- High error rate ($5xx > 5\%$ for 2 minutes)
- High latency ($p95 > 500ms$)
- Low traffic anomaly ($requests/sec < 1$)

13 Deployment Architecture

13.1 Docker Containers

Service	Base Image	Port
API	python:3.11-slim	8000
Frontend	nginx:alpine	8082
Prometheus	prom/prometheus:v2.54.1	9090
Grafana	grafana/grafana:11.1.0	3000
MLflow (optional)	custom	5000

13.2 Docker Compose Orchestration

File: `deployment/docker-compose.yml`

Services and dependencies:

- API starts first (loads model)
- Frontend waits for API (`depends_on`)
- Prometheus scrapes API
- Grafana queries Prometheus

13.3 Volume Mounts

Volume	Mount Point	Purpose
artifacts/	/app/artifacts	Load model + metadata
grafana_data/	/var/lib/grafana	Persist Grafana config
mlflow_data/	/mlflow/	Persist MLflow runs

13.4 Startup Steps

1. Clone repository
2. Activate venv: `source .venv/bin/activate`
3. Install dependencies: `pip install -r requirements.txt`
4. Generate artifacts: `python -m src.pipelines.run_model_workflow ...`
5. Start services: `cd deployment && docker-compose up -build`
6. Access services:
 - API: `http://localhost:8000`
 - Frontend: `http://localhost:8082`
 - Prometheus: `http://localhost:9090`
 - Grafana: `http://localhost:3000` (admin/admin)

14 Testing and CI/CD

14.1 Test Structure

```
tests/  
    unit/  
        test_ids.py  
        test_preprocess.py  
    integration/  
        test_api.py  
        test_model_loading.py  
    data/  
        test_validation.py  
    model/  
        test_pipeline.py
```

14.2 Coverage Gate

Target: 80% minimum

```
pytest -q --cov=src --cov-report=term-missing --cov-fail-under=80
```

Status: Passing (coverage > 80%)

14.3 CI/CD Pipeline

Workflow: .github/workflows/ci.yml

Steps:

1. Setup Python 3.11
2. Install dependencies
3. Run tests with coverage gate
4. Build Docker images
5. Run integration tests

Trigger: Every push and pull request to main

15 Responsible AI

15.1 Fairness and Bias

15.1.1 Dataset Limitations

- **Missing protected attributes:** No age, gender, ethnicity, income
- **Implication:** Cannot measure demographic parity or equalized odds
- **Alternative:** Fairness via proxy slices (Amount quantiles, Time buckets)

15.1.2 Proxy Slice Analysis

Slice	Definition	Metrics
Amount Quantiles	Q1, Q2, Q3, Q4	PR-AUC, precision, recall
Time Buckets	Hour-of-day from Time	PR-AUC, precision, recall
Combined	Amount × Time	Same across slices

15.2 Explainability

Method: SHAP (SHapley Additive exPlanations)

For Baseline (Logistic Regression):

- Feature coefficients (linear model)

- Output: Coefficient importance table

For Improved Model (LightGBM):

- SHAP values
- Output: `artifacts/figures/shap_summary.png`

Key Limitations:

- SHAP is model-level, not causal
- Cannot infer “V5 causes fraud” from feature importance
- Global importance is averaged across all instances

15.3 Privacy

Data Minimization:

- API accepts feature vector only (no PII)
- Dataset uses anonymized PCA features
- No customer identifiers in artifacts

Logging Policy (Recommended):

- Log: `request_id`, `model_version`, `latency`, `status`
- Log: aggregated metrics (counts, averages)
- Do NOT log: raw feature vectors
- Do NOT log: individual transaction scores

16 Security and Privacy Considerations

16.1 Current Protections

API:

- Input validation via Pydantic schemas
- CORS configured for localhost only
- No PII in input
- Generic error messages (no stack traces)

Data:

- No credentials in code (env vars)
- Features anonymized (PCA-transformed)
- Artifacts versioned (immutable)

16.2 Missing Controls (Future Work)

Control	Current	Recommendation
Authentication	None	Add JWT or API key
Rate limiting	None	Add slowapi library
TLS/HTTPS	Local only	Use valid certificates
Model signing	None	Sign joblib artifacts
Audit logging	None	Log with timestamp, version

17 Risks and Limitations

17.1 Model Risks

Risk	Severity	Cause	Mitigation
Class Imbalance	Medium	0.17% fraud rate	Class weight, PR-AUC
Concept Drift	High	Random split	Manual monitoring
Overfitting	Medium	Threshold on val	Test set for final eval
Lack of Calibration	High	No post-processing	Documented, not promised

17.2 System Risks

Risk	Current	Impact
No Authentication	Demo only	Unsuitable for internet
No Rate Limiting	Demo only	Vulnerable to DoS
Stateful Streaming	Simulation	Stream lost on restart
Single Artifact	No A/B testing	Slow rollout
No Automated Retrain	Manual	Model staleness grows

17.3 Production Gaps

- No authentication/rate limiting
- No drift detection
- Random split (not time-aware)
- Simulation streaming (not production)
- No probability calibration
- But clearly documented as future work

18 Acceptance Criteria

18.1 Success Indicators

Criterion	Status
Model PR-AUC 0.75 on test	0.769
API endpoints correct	Verified
Error handling 422/503	Verified
Docker Compose starts	Verified
Prometheus metrics exported	Verified
Grafana dashboards display	Verified
Frontend connects to API	Verified
80% test coverage	Achieved
Documentation complete	Complete
CI/CD pipeline running	Running

19 Conclusion

19.1 System Summary

This end-to-end fraud detection system demonstrates:

1. **Data-driven decision-making:** Raw transactions → ML scoring → tiered actions
2. **Operational awareness:** Capacity-driven thresholds matched to business constraints
3. **Observability:** Prometheus metrics and Grafana dashboards for system health
4. **Reproducibility:** Versioned artifacts, seeded processes, CI/CD automation
5. **Responsible AI:** Fairness analysis, explainability, privacy considerations

19.2 Key Achievements

- Baseline vs Improved models: Trained, compared, selected
- Threshold Strategy: Explicit top-K policy tied to review capacity
- API Design: Clean FastAPI with Pydantic validation
- Frontend: Real-time dashboard with decision visualization
- Monitoring: Prometheus metrics and Grafana dashboards
- Testing: 80% code coverage with CI/CD automation
- Deployment: Full Docker Compose stack ready for deployment
- Documentation: Comprehensive architecture and design docs

19.3 Design Principles Upheld

1. **Separation of Concerns:** Modular architecture (data, ML, serving, observability)
2. **Explainability:** SHAP importance, clear API contracts
3. **Accountability:** Version tracking, threshold logging, metric reporting
4. **Reproducibility:** Seeded processes, artifact versioning, CI/CD automation
5. **Honest Limitations:** Clear docs on uncalibrated scores, production gaps

19.4 Scope Assessment

For Demo/Academic Use: Complete and rigorous

For Production Deployment: Requires hardening in:

- Security (authentication, rate limiting, TLS)
- Monitoring (drift detection, label feedback)
- Scalability (Kubernetes, load balancing)
- Governance (audit logging, secrets management)

19.5 Final Recommendation

Suitable for:

- Academic projects and coursework
- Internal PoCs and demos
- Reference implementation for ML systems
- Educational use

With recommended production improvements, this system could evolve into a production-grade fraud detection service.

A Repository File Manifest

```
Final_Project_DMM501_Group1/
  src/                # Source code (data, ML, API, monitoring)
  frontend/           # Web dashboard (HTML/CSS/JS)
  deployment/         # Docker, Compose, Prometheus, Grafana
  tests/              # Unit/integration tests
  artifacts/          # Model outputs, metrics, figures
  data/               # Datasets
  .github/workflows/  # CI/CD (GitHub Actions)
  latex/              # LaTeX reports and documentation
```

B Acronyms and Definitions

Term	Definition
PR-AUC	Precision-Recall Area Under Curve
ROC-AUC	Receiver Operating Characteristic AUC
SHAP	SHapley Additive exPlanations
PCA	Principal Component Analysis
CI/CD	Continuous Integration / Continuous Deployment
TLS	Transport Layer Security
JWT	JSON Web Token